

HTML 구조를 고려한 계층적 청킹 및 하이브리드 검색을 통한 한국어 기술 블로그 RAG 성능 최적화

Optimizing Korean Tech RAG Performance via HTML-Aware Hierarchical Chunking and Hybrid Search

요약

본 연구는 구조적 복잡성이 높은 한국어 기술 블로그에 콘텐츠가 동적 추가될 때 이를 실시간으로 반영할 수 있는 자동화된 RAG(Retrieval-Augmented Generation) 파이프라인 최적화 방안을 제안한다. 기존의 단순 텍스트 분할 방식은 코드 블록, 표, 헤더 등 웹 문서 특유의 구조를 보존하지 못해 문맥적 무결성을 훼손한다는 한계가 있다. 이를 해결하기 위해 HTML 헤더 기반의 계층적 청킹과 특수 블록(표, 코드) 분리 전략을 적용하였다.

실험 결과, 제안하는 HTML 구조 기반 청킹과 하이브리드 검색을 통합한 파이프라인은 기존 단순 방식 대비 Faithfulness (0.747→0.851) 및 Answer Relevancy (0.749→0.848) 지표에서 유의미한 성능 향상을 입증하였다.

1. 서론

최근 많은 기업과 개발자 그룹은 기술적 전문성을 홍보하고 브랜드 가치를 제고하기 위해 기술 블로그를 적극 활용하고 있다. 이러한 웹 콘텐츠는 기업의 핵심 지식 자산이며, 이를 효과적으로 탐색할 수 있는 AI 챗봇의 도입은 정보 접근성 극대화 및 사용자 이탈률 감소를 통해 서비스 경쟁력 강화에 기여한다. 그러나 대규모 언어 모델(LLM)은 뛰어난 생성 능력에도 불구하고, 최신 정보 접근의 한계와 환각(Hallucination)[1] 현상으로 인해 실시간 데이터가 중요한 기술 서비스에 적용하기에는 제약이 따른다.

이러한 한계를 극복하기 위해 답변의 정확성을 높이는 검색 증강 생성(RAG) 기술[2]이 대안으로 부상했으나, 기술 블로그와 같은 동적 웹 콘텐츠에 적용할 때는 새로운 문제가 발생한다. 기술 블로그는 단순 텍스트 외에도 표(<table>), 코드 블록(<pre>) 등 정형화된 HTML 구조를 포함하며, 실시간으로 새로운 게시물이 생성되는 동적인 특성을 가진다. 따라서 RAG 시스템이 최신성을 유지하면서도 정보를 정확히 추출하기 위해서는, 새로운 콘텐츠를 탐지하고 구조화된 데이터를 해석하여 인덱싱하는 자동화된 파이프라인 아키텍처가 필수적이다

기존의 단순 텍스트 분할 방식은 HTML의 논리적 구조를 무시하여 코드 단절이나 문맥 파괴를 초래하며, 이는 검색 정밀도와 답변 품질(Faithfulness)의 저하로 이어진다. 최근 HTML 구조 자체를 활용하는 HtmIRAG[1] 등의 연구가 진행되었으나, 이는 주로 영어권 일반 웹 문서에 국한되어 있어 한국어 기술 블로그 도메인에 특화된 연구는 미비한 실정이다.

본 연구에서는 한국어 기술 블로그 최적화를 위해 HTML 헤더 기반 계층적 청킹과 특수 블록(코드/표) 분리 전략을 제안한다. 또한 하이브리드 검색, Cross-Encoder 재정

렬, 한국어 임베딩 모델 최적화를 통해 신규 콘텐츠를 자동으로 학습하는 효율적인 RAG 파이프라인 구축 방안을 제시한다.

주요 기여를 정리하면 다음과 같다.

- **구조적 문맥 보존을 위한 전처리 전략** : HTML 헤더 및 특수 블록(코드·표) 분리 전략을 통해 기술 문서 특유의 논리 구조를 보존함으로써, 코드 단절 문제를 해결하고 검색의 문맥적 무결성을 확보하였다.

- **자동화 파이프라인 아키텍처 설계** : 신규 포스트 발행부터 파싱, 인덱싱까지 전 과정을 자동화하여 정보의 최신성이 중요한 기술 블로그 서비스의 실무적 적용 가능성을 높였다.

2. 제안 방법

데이터 수집부터 최종 생성에 이르는 전 과정을 자동화한 RAG 파이프라인을 제안한다. 특히 기술 블로그의 구조적 특성을 반영한 데이터 전처리 및 검색 성능 최적화에 주안점을 둔다.

2.1 HTML 구조 기반 계층적 청킹

단순 텍스트 재귀 분할 방식은 코드 블록이나 표와 같은 HTML 요소의 문맥을 파괴하기에 답변의 신뢰성을 저하시킨다. 이를 해결하기 위해 다음과 같은 전략을 적용한다.

2.1.1 특수 블록 분리 및 노이즈 제거

BeautifulSoup을 활용해 불필요한 태그를 제거한 후, 코드(<pre>)와 표(<table>)를 일반 텍스트와 분리하여 추

출한다. 이는 분할 과정에서 구조적 데이터가 절단되는 논리적 오류를 방지함으로써, 검색 시 관련 없는 파편이 섞이는 정보 노이즈를 최소화하기 위함이다.

2.1.2 계층적 섹션 분할

HTMLHeaderTextSplitter를 활용해 h1~h4 헤더를 기반으로 문서를 1차 분할한다. 작성자가 의도한 논리적 단위를 유지함으로써 관련 정보가 동일 청크에 포함될 확률을 높여 문맥적 무결성(Context Integrity)을 보존한다. 2차로 RecursiveCharacterTextSplitter를 적용해 최적 크기인 400자(오버랩 100자)로 세분화한다. 마지막으로 분리된 각 청크에 post_id, chunk_type(text/code/table) 등의 메타데이터를 부여하여 검색 단계에서의 식별력을 강화한다.

2.2 하이브리드 검색 및 재정렬

기술 용어의 정밀 탐색을 위해 키워드 기반의 BM25와 문맥적 유사성을 포착하는 Dense Embedding 검색을 결합한다. 이는 고유 명사 적중률이 높은 키워드 검색과 질의 의도를 파악하는 의미 검색의 상호 보완(Semantic + Keyword)을 통해 검색 재현율을 극대화하기 위함이다.

실험을 통해 도출된 가중치 (Embedding 0.65, BM25 0.35)로 합산된 후보군을 mxbai-rerank-large-v1 모델로 재정렬하여 최적의 Top-4 문서를 선정한다.

2.3 시스템 구현 예시

Fig1은 본 연구에서 제안한 자동화 파이프라인이 실제 기술 블로그 환경에 통합되어 구현된 챗봇 인터페이스이다. 사용자가 포스트를 열람하며 특정 기술 개념에 대해 질문을 입력하면, 시스템은 Vector DB 내 임베딩 데이터를 참조하여 관련 문맥을 신속하게 검색하고, 하이브리드 검색 및 재정렬 과정을 거쳐 최적화된 정보를 바탕으로 답변을 생성한다. 이러한 구현 예시는 제안된 아키텍처를 실제로 가시화한 것이며, 기술 문서에 대한 본 시스템의 실무적 적용 가능성을 입증한다.

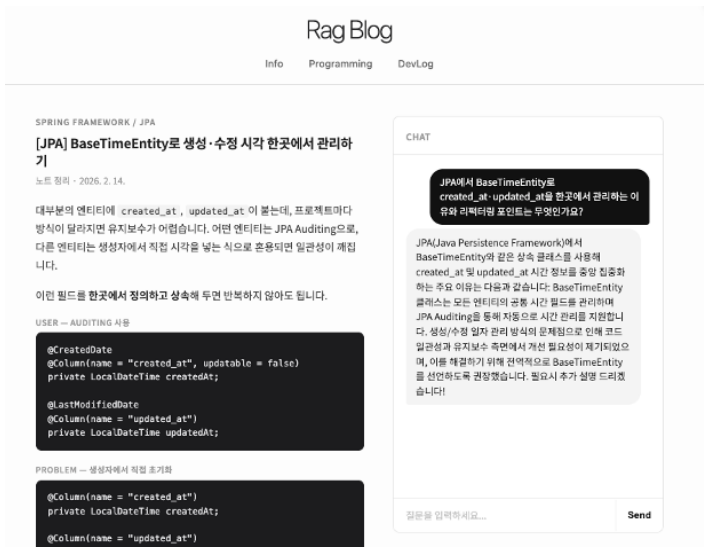


Fig1. Implementation of RAG Chatbot for Tech Blog

3. 실험 및 결과

3.1 실험 설정

375개의 프로그래밍 관련 기술 블로그 포스트를 3,217개의 청크로 분할하여 실험을 진행하였다. 시스템의 질적 성능 검증을 위해 실제 사용자 의도와 합성 데이터를 결합하여 총 20종의 평가 질의 셋을 구성하였으며, 상세한 구축 방법은 3.5절에서 기술한다.

3.2 청킹 전략 비교

제안하는 HTML 구조 기반 청킹의 유효성을 검증하고자, RecursiveCharacterTextSplitter를 사용하는 기존 방식(Baseline)과 RAGAS 성능을 비교하였다. 두 방식 모두 동일한 임베딩 모델과 하이퍼 파라미터를 적용하여 변수를 통제하였으며, 결과는 [Table1]과 같다.

Table 1: 청킹 전략 비교 평가 (RAGAS)		
청킹 전략	Faithfulness	Answer Relevancy
단순 분할 (Baseline)	0.832	0.843
HTML 계층적 청킹 (Proposed)	0.852	0.848

3.3 임베딩 모델 비교

제안하는 파이프라인 성능에 핵심적인 영향을 미치는 임베딩 모델 3종(nomic-embed-text, mixedbread-ai/mxbai-embed-large-v1, bge-m3)을 대상으로 비교 평가를 수행하였다.

모든 하이퍼 파라미터를 3.4절의 최적값으로 고정된 뒤 임베딩 모델에 따른 성능 변화를 측정하였으며, 그 결과는 [Table2]와 같다.

Table 2: 임베딩 모델 비교 평가 (RAGAS)		
Model	Faithfulness	Answer Relevancy
BGE-M3	0.849	0.846
nomic-embed-text	0.834	0.847
mxbai-embed-large-v1	0.851	0.844

실험 결과, mixedbread-ai/mxbai-embed-large-v1 모델이 Faithfulness 0.851 에서 가장 우수한 성적을 기록하였다. 이에 따라 본 연구에서 제안하는 RAG 파이프라인의 최종 모델로 해당 모델을 선정하였다.

3.4 하이퍼 파라미터 최적화

안정적인 성능 확보를 위해 주요 하이퍼 파라미터를 다음과 같이 최적화하였다.

- 청크 크기 : 200, 400, 800자를 비교 실험한 결과, 200자는 문맥 손실을 초래했고, 800자는 과도한 정보 포함으로 검색 정밀도(precision)가 저하되었다. 이에 따라 문맥 유지와 정밀도의 균형점인 400자(오버랩 100자)를 최적값으로 선정하였다.

- 하이브리드 가중치 : 임베딩과 BM25 점수의 비율을 0.5~0.7 범위에서 조정해 결과, 임베딩 0.65, BM25 0.35일 때 가장 우수한 성능을 보였다.

3.5 평가 데이터 셋 구축 및 설계

RAG 시스템의 실무적 유효성을 검증하기 위해 실제 사용자 데이터와 LLM 기반 합성 데이터를 결합한 평가 셋을 구축하였다. 특히 실제 사용자의 의도가 담긴 댓글 및 Q&A 데이터를 적극 반영하여 실무적 변별력을 높였으며, 데이터 다양성 확보를 위해 Gemini 3.0 Pro를 활용한 질의 생성을 병행하였다. 수집 및 생성된 모든 질의는 연구자의 직접 검수를 거쳐 논리적 타당성이 확보된 최종 20종을 선정하였으며, 그 예시는 [Table3]과 같다.

Table 3: 평가 데이터 셋 질의 예시

ID	질의 내용
Q1	RAGAS로 RAG 파이프라인을 평가할 때 필요한 데이터 셋 구성요소와 Ground Truth의 역할은 무엇인가요?
Q2	Docker exec와 docker run의 차이는 무엇이고, 실행 중인 컨테이너에 명령을 넣을 때 왜 docker exec를 쓰나요?
Q3	JPA Auditing에서 @EnableJpaAuditing과 @EntityListeners는 무엇을 하나요?

3.6 LLM 모델 비교 평가

RAG 시스템의 최종 생성 품질을 측정하기 위해 RAGAS 지표 (Faithfulness, Answer Relevancy)를 기반으로 LLM 간 성능을 비교하였다. 변수 통제를 위해 임베딩 모델은 가장 우수한 성능을 보인 mx-bai-embed-large-v1으로 고정하였으며, 오픈소스 모델인 EXAONE-4.0-1.2B와 상용 모델 2종(GPT-4o, GPT-4o-mini)을 비교 군으로 선정하였다.

Table 4: LLM 모델 비교 평가 (RAGAS)

Model	Faithfulness	Answer Relevancy
Exaone4.0-1.2b	0.801	0.874
Gpt-4o	0.812	0.879
Gpt-4o-mini	0.802	0.877

실험 결과 1.2B 파라미터의 소형 모델인 EXAONE-4.0-1.2B가 GPT-4o와 대등한 생성 품질을 기록하였다. 이는 본 연구에서 제안한 HTML 기반 전처리와 하이브리드 검색 전략이 구조적 노이즈를 제거하고 검색 문맥의 순도를 높여, 모델의 추론 부담을 효과적으로 낮췄음을 시사한다. 결과적으로 정교한 파이프라인 설계가 뒷받침될 경우, 소형 오픈소스 모델만으로도 경제성을 확보한 상용급 RAG 시스템 구축이 가능함을 입증하였다.

3.7 Retrieval 방식 비교 실험

검색 알고리즘 구성에 따른 정밀도 검증을 위해 (1) Dense Retrieval, (2) Hybrid Retrieval, (3) Hybrid Reranker 세 가지 방식을 비교 평가하였다. 각 방식에 따른 RAGAS 평가 결과는 [Table5]와 같다.

실험 결과, 임베딩과 BM25를 결합하고 Cross-Encoder 재정렬을 거치는 Hybrid Reranker 방식이 모든 지표에서 가장 우수한 성능을 기록하였다. 특히 단순 Dense Retrieval 방식과 비교했을 때 Answer Relevancy 지표에서 약

13.2%의 유의미한 성능 향상을 보여, 본 연구에서 제안하는 검색 전략의 유효성을 입증하였다.

Table 5: Retrieval 방식 비교 실험 (RAGAS)

Retrieval	Faithfulness	Answer Relevancy
Dense Retrieval	0.746	0.749
Hybrid Retrieval	0.777	0.796
Hybrid Reranker	0.794	0.848

4. 결론

본 연구는 한국어 기술 블로그 콘텐츠가 동적으로 추가될 때 이를 자동으로 학습하고 임베딩하는 RAG 시스템 최적화 방안을 제안하였다. 실험 결과, HTML 구조(헤드·코드·표)를 고려한 청킹 전략이 단순 텍스트 분할 방식 대비 Faithfulness 및 Answer Relevancy를 향상시켜 문맥 보존에 유리함을 입증하였다.

이는 기술 문서나 웹페이지와 같이 명확한 구조를 가진 동적 데이터를 실시간으로 RAG에 활용할 경우, 콘텐츠의 구조적 특성을 반영한 전처리가 성능 향상의 핵심 요소임을 시사한다. 본 연구의 방법론은 유사한 HTML 구조를 가지는 뉴스, 쇼핑몰 등 타 도메인의 RAG 시스템 구축에도 실질적인 가이드라인을 제공할 것으로 기대된다.

향후 연구에서는 Faithfulness와 Answer Relevancy를 극대화하기 위해 의미론적 분석을 반영한 청킹 전략의 고도화를 추진하고, 질의 셋 확대를 통해 시스템의 일반화 성능을 다각도로 검증할 계획이다. 특히 기술 블로그 내의 도표나 시각 정보를 텍스트와 함께 이해하고 추론할 수 있도록 이미지 인식 기능을 결합한 멀티 모달 (Multimodal) RAG 파이프라인으로의 확장을 도모하고자 한다. 최종적으로는 제안된 시스템을 실제 운영 중인 웹사이트에 적용하여, 챗봇 도입 전후의 사용자 페이지 이탈률 및 체류 시간 등 주요 지표 변화를 정밀하게 비교 측정함으로써 본 연구가 제시하는 파이프라인의 실무적 유용성과 서비스 기여도를 객관적으로 입증할 예정이다.

부 록

참 고 문 헌

- [1] Ji, Ziwei, et al. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1-38.
- [2] Lewis, Patrick, et al. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9475.
- [3] Z. Chen et al., "HtmlRAG: HTML is better than Raw Text for LLMs," *arXiv preprint arXiv:2411.02959*, 2024.